



Git Guide.

Every Git and GitHub answer in one place.

Ask in plain language, or paste the error Git printed.
Exact commands, a danger level for each, and its undo.

an undo for everything · works offline · no trackers · MIT



Amey Thakur

amey-thakur.github.io/GIT-GUIDE



Every Git and GitHub answer in one place.

A note from Amey

Git carries nearly every codebase on earth, and almost everyone who uses it learned it by accident: a command from a teammate, a midnight search, a ritual repeated without understanding.

Git Guide exists to end that way of learning. Ask in plain language, or paste the error Git printed, and get the exact commands with the two things most resources never give: how dangerous each one is, and how to take it back.

These pages hold the whole of it: the model that replaces memorizing, every command section, every error decoded, and the chapters on GitHub and on how teams work. The living version, searchable and always current, waits at **amey-thakur.github.io/GIT-GUIDE**.

Amey



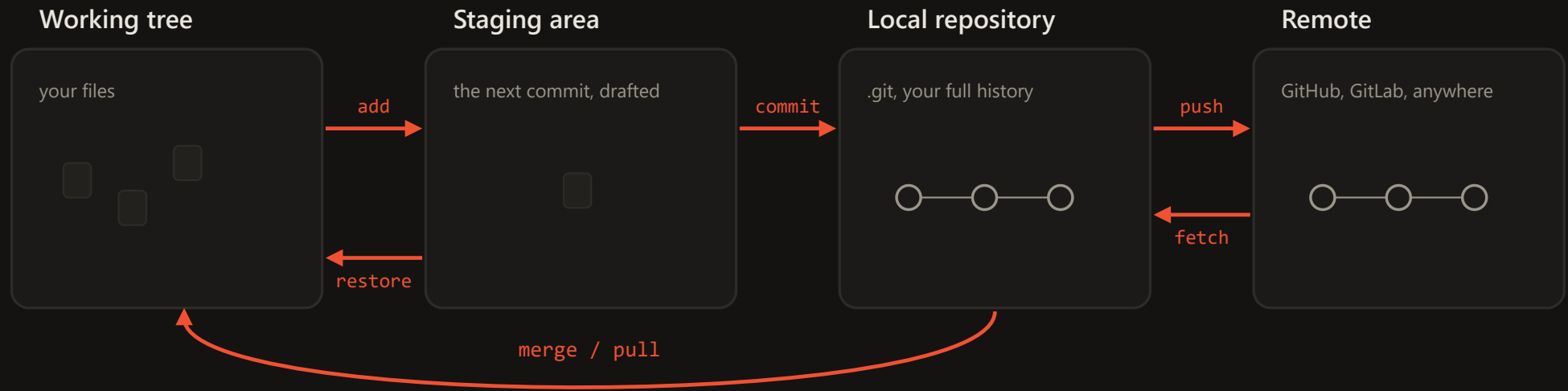
Amey Thakur

amey-thakur.github.io/GIT-GUIDE



Every Git and GitHub answer in one place.

Where your code lives



add copies a change into the staging area, the draft of your next commit. You choose what goes in.

push uploads your commits; **fetch** downloads theirs and stops. **pull** is fetch plus merge.

commit seals the draft into a permanent snapshot in your local repository. Still only on your machine.

restore copies recorded versions back over your files; **reset** moves the branch pointer itself. That is where history is un-happened.



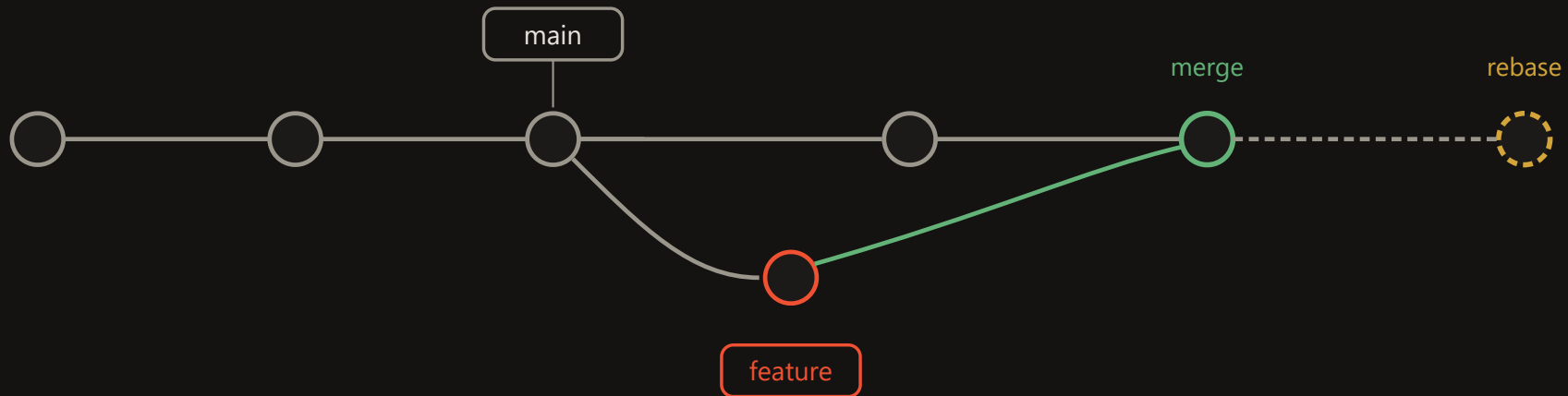
Amey Thakur

amey-thakur.github.io/GIT-GUIDE



Every Git and GitHub answer in one place.

Commits, branches, HEAD



A **branch** is a movable label on one commit. Creating one copies nothing; it is instant and free.

A **rebase** replays your commits as new ones for a straight line. It rewrites history: your own unshared work only.

A **merge** commit has two parents and joins the lines. History shows what truly happened; always safe on shared branches.

HEAD is where you stand, normally attached to a branch. Standing on a commit directly is detached HEAD: a state, not an error.



From zero to first push

1 Install

Windows: `winget install Git.Git` · macOS: `brew install git` · Debian: `sudo apt install git`

```
git --version
```

3 One key, every host

Add the public key to GitHub, GitLab, Bitbucket, Azure: the same key works on all

```
ssh-keygen -t ed25519 -C "<email>"
```

5 The loop you will run forever

Edit, stage, commit, push. Everything else is a variation

```
git add <file> && git commit -m "<message>" && git push
```

2 Identity

The same email as your GitHub account, or commits will not count on your profile

```
git config --global user.name "<name>" && git config --global user.email "<email>"
```

4 First repository

Joining: `git clone <url>` · Publishing a folder of yours:

```
gh repo create <name> --public --source . --push
```

6 When it goes wrong

It will, for everyone. Ask the site in plain language, or paste the exact error

```
amey-thakur.github.io/GIT-GUIDE
```



Setup and identity

```
git config --global user.name "<name>"
```

Name stamped into your commits

```
git config --global user.email "<email>"
```

Use your host account email so commits link to your profile

```
git config --global init.defaultBranch main
```

New repositories start on main

```
git config --global core.editor "code --wait"
```

Commit messages open in VS Code

```
git config --global pull.ff only
```

Pull never creates surprise merge commits

```
git config --global fetch.prune true
```

Deleted remote branches disappear locally too

```
git config --global rebase.autoStash true
```

Pull with rebase works even with edits in progress

```
git config --global core.autocrlf true
```

Windows line endings handled once; macOS and Linux use input

```
git config --list --show-origin
```

Every setting and the file it comes from

Start a project

```
git init -b main
```

Turn this folder into a repository

```
git clone <url>
```

Copy a repository, any host, full history

```
git clone --depth 1 <url>
```

Latest snapshot only, fastest

```
git remote add origin <url>
```

Connect a local repository to a host

```
git push -u origin main
```

First push; -u links the branches for good



The daily loop

```
git status -sb
```

What changed, one line per file

```
git diff
```

Unstaged edits, line by line

```
git diff --staged
```

What the next commit will contain

```
git add <file>
```

Stage a change; git add -p picks piece by piece

```
git commit -m "<message>"
```

Seal the staged snapshot

```
git commit --amend --no-edit
```

Add staged changes to the last commit

```
git fetch
```

See what the remote has before touching anything

```
git pull --rebase
```

Get remote commits, replay yours on top

```
git push
```

Publish your commits

Branches

```
git switch -c <branch>
```

Create and move in one step

```
git switch <branch>
```

Move to an existing branch

```
git switch -
```

Back to the branch you just left

```
git branch --show-current
```

Where am I

```
git branch -vv
```

Every local branch with its remote and ahead-behind state

```
git branch -a
```

Every branch, local and remote

```
git push -u origin <branch>
```

Publish a new branch

```
git branch -d <branch>
```

Delete a merged branch; -D forces

```
git push origin --delete <branch>
```

Delete it on the remote too

```
git branch -m <old> <new>
```

Rename



Merge and rebase

```
git merge <branch>
```

Bring a branch in; history stays true

```
git merge --no-ff <branch>
```

Always keep the feature visible as one bubble

```
git rebase main
```

Replay your branch on top of main

```
git rebase -i HEAD~<n>
```

Rewrite, reorder, squash your last n commits

```
git cherry-pick <hash>
```

Copy one commit onto this branch

```
git merge --abort
```

Back out of a conflicted merge

```
git rebase --continue
```

Resume after resolving a conflict

Undo and rescue

```
git restore <file>
```

Discard edits, back to last commit

```
git restore --staged <file>
```

Unstage, keep the edits

```
git commit --amend -m "<new message>"
```

Rewrite the last commit message

```
git reset --soft HEAD~1
```

Uncommit, keep everything staged

```
git reset --hard HEAD~1
```

Erase the last commit and its work

```
git revert <hash>
```

Undo by adding an opposite commit; safe when pushed

```
git reflog
```

Every state you have been in; the safety net

```
git reset --hard ORIG_HEAD
```

Undo the merge, rebase, or pull that just happened

```
git clean -nd
```

Preview deleting untracked files; -fd deletes



Stash

```
git stash push -u -m "<label>"
```

Set everything aside, untracked included

```
git stash list
```

What is shelved

```
git stash pop
```

Bring the newest back and drop it

```
git stash apply stash@{<n>}
```

Bring a specific one back, keep it shelved

```
git stash show -p
```

See inside before applying

Inspect and search

```
git log --oneline --graph --all
```

The whole history as a graph

```
git log -p -- <file>
```

Every change to one file; --follow crosses renames

```
git log --grep="<text>"
```

Find commits by message

```
git log -S "<text>"
```

Find commits that added or removed this text

```
git diff main...<branch>
```

What the branch changed since it split; the pull request view

```
git shortlog -sn --all
```

Commits per author, ranked

```
git blame -w <file>
```

Who last touched each line, ignoring whitespace

```
git show <hash>
```

One commit in full; --stat for files only

```
git bisect start
```

Binary-search history for the commit that broke it

```
git branch -a --contains <hash>
```

Which branches carry this commit



Tags and releases

```
git tag -a v1.0.0 -m "<note>"
```

Mark a release

```
git push origin v1.0.0
```

Tags do not push themselves

```
git tag -d <tag>
```

Delete locally; push origin --delete <tag> for the remote

```
git describe --tags
```

Nearest release to where you stand

```
git switch --detach v1.0.0
```

Visit a release; switch - to come back

Files and ignoring

```
printf "node_modules/\n" >> .gitignore
```

Ignore by pattern; affects untracked files only

```
git rm --cached <file>
```

Stop tracking, keep on disk

```
git mv <old> <new>
```

Rename and stage in one step

```
git update-index --skip-worktree <file>
```

Ignore your local edits to a tracked file

```
printf "* text=auto\n" >> .gitattributes
```

Line endings settled for the whole team, committed once

```
touch <folder>/.gitkeep
```

Keep an empty folder



Every platform, same Git

Git is identical everywhere; only the remote URL and sign-in differ. The same SSH public key works on every host.

```
git clone git@github.com:<user>/<repo>.git
```

GitHub over SSH

```
git clone git@gitlab.com:<user>/<repo>.git
```

GitLab

```
git clone git@bitbucket.org:<workspace>/<repo>.git
```

Bitbucket

```
git clone https://dev.azure.com/<org>/<project>/_git/<repo>
```

Azure DevOps (Azure Repos)

```
git clone https://git-codecommit.  
<region>.amazonaws.com/v1/repos/<repo>
```

AWS CodeCommit

```
git config --global credential.helper manager
```

Browser sign-in for HTTPS remotes, all hosts

```
git remote set-url origin <url>
```

Move between hosts or between HTTPS and SSH

```
git clone --mirror <old> && git push --mirror <new>
```

Migrate anywhere with full history

No host at all

Git predates the hosts and needs none of them.

```
git init --bare ~/server/repo.git
```

Your own remote on any machine or shared drive

```
git bundle create repo.bundle --all
```

The whole repository as one file: USB, email, air gap

```
git clone repo.bundle -b main <folder>
```

Clone from the file, no network

```
git format-patch -3
```

Last three commits as mailable patch files

```
git am <file>.patch
```

Apply mailed patches, authorship intact

```
git archive --format=zip -o src.zip HEAD
```

Clean source export, no .git



History surgery

Everything here rewrites history. Fresh clone first, backup kept, force-with-lease after, teammates re-clone.

```
git commit --fixup <hash>
```

Mark a fix as belonging to an earlier commit

```
git rebase -i --autosquash <hash>^
```

Melt fixups into their targets

```
git filter-repo --invert-paths --path <file>
```

Erase a file from all history

```
git filter-repo --strip-blobs-bigger-than 10M
```

Erase every giant blob

```
git push --force-with-lease
```

The only force push worth typing

Scale and speed

```
git clone --filter=blob:none --sparse <url>
```

Huge repository, download almost nothing

```
git sparse-checkout set <folders>
```

Materialize only what you work on

```
git worktree add ../<repo>-review <branch>
```

Second folder, second branch, zero stashing

```
git lfs track "*.psd"
```

Large binaries stored properly

```
git count-objects -vH
```

How heavy is this repository really

```
git maintenance start
```

Background upkeep keeps big repositories fast



Submodules

```
git submodule add <url> <path>
```

Pin another repository at an exact commit inside yours

```
git clone --recurse-submodules <url>
```

Clone a repo and its submodules in one step

```
git submodule update --init --recursive
```

Fill the empty folders after a plain clone

```
git submodule update --remote
```

Move pins to each submodule's latest commit

Quality of life

```
git config --global help.autocorrect prompt
```

git statsu asks: run git status instead?

```
git config --global alias.st "status -sb"
```

git st forever

```
git config --global alias.lg "log --oneline --graph --all"
```

git lg: the graph

```
git config --global rerere.enabled true
```

Never resolve the same conflict twice

```
git config --global gpg.format ssh
```

First of three lines to signed, Verified commits

```
git commit --allow-empty -m "rerun"
```

Retrigger CI without touching code

```
git config core.hooksPath .githooks
```

Versioned hooks the whole team shares



How teams run Git

> Working alone

Commit at every working state; push is your backup; branch before experiments

```
git add -A && git commit -m "<message>" && git push
```

> Fork

For repositories you cannot push to, which is all of open source

```
gh repo fork <owner>/<repo> --clone
```

> Releases and hotfixes

A release is a tag; a hotfix is a branch from that tag, merged back after shipping

```
git switch -c hotfix-2.1.1 v2.1.0
```

> Feature branch

The team default: main stays releasable, every change arrives by pull request

```
git switch -c <change> && git push -u origin <change>
```

> Trunk-based

Branches live hours, not weeks; tiny changes merge daily behind flags

```
git config --global pull.rebase true
```

The rules that keep teams safe. Never rewrite a branch others build on; rewrite your own freely with force-with-lease. Pull with rebase. Protect main so nothing lands without review and green checks. When anything is unclear: git status names the state you are in, and usually the way out.



Your first repository

Name

Short, lowercase, hyphens: weather-app

Add a README

The front page, and your first commit

Add .gitignore

Pick your language; noise stays out from day one

Choose a license

MIT if unsure; none means all rights reserved

The pull request path

branch · commit · push · open the PR · checks run · review answers with commits · merge, and the branch retires.

`gh pr create --fill`

Open it from the terminal

`gh pr checkout <number>`

Review one locally

`gh pr merge --squash --delete-branch`

Land it clean

The words nobody explains

repository a project and its entire history; everyone says repo

README.md the front page, rendered automatically

Markdown plain text with light marks; GitHub renders it everywhere

.gitignore what Git must never track

LICENSE the legal terms of reuse

public, private everyone reads, only you write; or invitation only

star a bookmark and a thank-you; stars rank repositories

watch subscribe to issues, PRs, and releases

clone vs fork to your machine; to your account

issue a tracked conversation about one bug or request

default branch usually main: what visitors see, what PRs target

gist one shareable file with its own URL

organization a shared account for a team



The errors you will meet

The message, then the cause. Every fix is one search away **on the site**.

**fatal: not a git repository (or any of the parent directories):
.git**

This folder, and none above it, contains a .git directory. You are outside the project.

remote: Repository not found.

The URL has a typo, the repository is private and you lack access, or you are authenticated as the wrong account.

git@github.com: Permission denied (publickey).

The host never received a key it recognizes: no key loaded, key not added to your account, or an HTTPS problem disguised by an SSH URL.

fatal: Authentication failed for 'https://github.com/...'

The credential your machine sent is wrong, expired, or a password where a token is required.

! [rejected] main -> main (non-fast-forward)

The remote has commits you do not have. Git refuses to overwrite them.

! [rejected] main -> main (fetch first)

Same cause as non-fast-forward: someone pushed before you.

error: src refspec main does not match any

The branch you are pushing does not exist locally, almost always because there is no commit yet.

fatal: refusing to merge unrelated histories

The two sides share no common commit, typically a fresh git init pulled against a remote that already has commits.

CONFLICT (content): Merge conflict in <file>

Both sides changed the same lines; Git needs a human decision.

**error: Your local changes to the following files would be
overwritten by merge**

The operation needs to rewrite files you have edited but not committed. Git refuses to destroy them silently.

fatal: The current branch has no upstream branch

This local branch has never been connected to a remote branch, so plain git push does not know where to go.

You are in 'detached HEAD' state.

Not an error. You checked out a commit or tag directly, so you stand on a snapshot instead of a branch.



The errors you will meet

The message, then the cause. Every fix is one search away **on the site**.

fatal: unable to auto-detect email address

Git has no identity to stamp into the commit.

error: cannot pull with rebase: You have unstaged changes

A rebase must start from a clean tree, and yours has edits.

fatal: unable to access '...': Could not resolve host: github.com

A network problem, not a Git problem: no internet, VPN or proxy in the way, or DNS down.

fatal: unable to access '...': SSL certificate problem

Something intercepts HTTPS, usually a corporate proxy with its own certificate that Git does not trust yet.

error: RPC failed; HTTP 408 curl 22 (or 400, or curl 55)

A large push choked, often against a proxy or server limit on request size.

remote: error: GH001: Large files detected.

GitHub refuses files over 100 MB in normal Git storage.

fatal: Unable to create '.git/index.lock': File exists.

Another Git process is running, or one crashed and left its lock behind.

fatal: detected dubious ownership in repository

The repository folder belongs to a different OS user than the one running Git, common in containers, WSL, and network drives.

warning: LF will be replaced by CRLF the next time Git touches it

Windows line endings meeting Unix line endings. Harmless, but configure it once and it goes quiet.

error: pathspec '<name>' did not match any file(s) known to git

The file or branch name does not exist as typed: a typo, wrong case, or a branch you have not fetched.

fatal: a branch named '<name>' already exists

You are creating a branch whose name is taken.

error: the branch '<name>' is not fully merged

git branch -d protects commits that exist nowhere else. This is the guardrail working.



The errors you will meet

The message, then the cause. Every fix is one search away **on the site**.

fatal: destination path '<folder>' already exists and is not an empty directory

git clone creates the folder, and a folder with that name is already there.

error: remote origin already exists

You are adding a remote named origin to a repository that already has one.

fatal: 'origin' does not appear to be a git repository

You pushed or pulled against a remote name that is not configured here.

Your branch and 'origin/main' have diverged

Both you and the remote gained different commits since you last synced.

Your branch is ahead of 'origin/main' by N commits

Not an error. You have local commits the remote lacks; push when ready.

fatal: bad object HEAD (or: loose object is corrupt)

The object database is damaged, usually a crash, a full disk, or a synced-folder service fighting over .git.

error: unable to create file <path>: Filename too long

Windows' 260-character path limit, hit by deep folder structures.

error: invalid path '<name>'

The repository contains a file name Windows forbids: reserved names like aux or con, or characters like colon and question mark, committed from Linux or macOS.

warning: You appear to have cloned an empty repository.

Not an error. The remote exists but has no commits yet.

fatal: this operation must be run in a work tree

You are inside the .git folder itself, or in a bare repository that has no working files by design.

fatal: tag '<name>' already exists

Tags are unique names; this one is taken.

error: cannot spawn .git/hooks/pre-commit: No such file or directory

A hook script is missing, not executable, or has Windows line endings breaking its shebang line.



The errors you will meet

The message, then the cause. Every fix is one search away **on the site**.

error: gpg failed to sign the data

Signing is switched on but the signing setup is incomplete or the key is unavailable.

error: Pulling is not possible because you have unmerged files.

An earlier merge stopped at conflicts and is still waiting for you to finish it.

hint: You have divergent branches and need to specify how to reconcile them.

Newer Git asks you to choose a pull strategy once instead of silently merging.

fatal: could not read Username for 'https://github.com': No such device or address

Git wanted to ask for credentials but no terminal was attached, the classic CI and script failure.

fatal: early EOF (the remote end hung up unexpectedly)

A large clone or fetch died mid-transfer: unstable connection, proxy timeout, or a server cutting long downloads.

xcrun: error: invalid active developer path (/Library/Developer/CommandLineTools)

A macOS update removed the command line tools, and git on a Mac lives inside them.

Could not open a connection to your authentication agent.

ssh-add needs the ssh-agent process, and none is running in this shell.

fatal: Not possible to fast-forward, aborting.

Your pull is set to fast-forward only, and the branches have diverged, so a decision is needed.

You are not currently on a branch.

Pull needs a branch to update, and you stand on a detached commit.

error: failed to push some refs to '<url>'

The wrapper line under most push rejections; the real reason is the line above it, usually non-fast-forward.

fatal: could not create work tree dir '<name>': Permission denied

You are cloning into a folder your user cannot write to.

error: insufficient permission for adding an object to repository database .git/objects

Files inside .git belong to another user, typically after running Git with sudo once or sharing a repo folder.



The errors you will meet

The message, then the cause. Every fix is one search away **on the site**.

error: cannot lock ref 'refs/remotes/origin/<branch>': unable to resolve reference

A stale or corrupted remote-tracking reference, often after an interrupted fetch.

error: cannot lock ref 'refs/heads/<x>/<y>': 'refs/heads/<x>' exists

Branch names are paths: a branch named x blocks any branch named x/y, and the reverse.

fatal: bad revision 'HEAD'

The repository has no commits yet, so HEAD points at nothing.

fatal: ambiguous argument 'HEAD': unknown revision or path not in the working tree.

Same family: an empty repository, or a name Git cannot tell apart from a file.

error: The following untracked working tree files would be overwritten by checkout

The branch you are switching to tracks files that exist here untracked; Git refuses to destroy them silently.

error: unable to unlink old '<file>': Permission denied

Something holds the file open: an editor, a running process, or an antivirus scan, most often on Windows.

fatal: '<branch>' is already checked out at '<path>'

Worktrees guarantee one checkout per branch, and that branch lives in another folder.

error: You need to resolve your current index first

A merge stopped at conflicts, and Git will not move on until you finish or abandon it.

error: commit <hash> is a merge but no -m option was given.

Reverting a merge needs to know which parent line survives.

fatal: bad config line <n> in file <path>

A typo in a config file: an unclosed quote, a stray character, or a section header gone wrong.

There is no tracking information for the current branch.

Pull does not know which remote branch feeds this one.

fatal: couldn't find remote ref <branch>

No branch by that name exists on the remote: a typo, a deleted branch, or main versus master.



The errors you will meet

The message, then the cause. Every fix is one search away **on the site**.

Host key verification failed.

SSH does not yet trust this server's fingerprint, or the stored one no longer matches.

ssh: connect to host github.com port 22: Connection timed out

The network blocks port 22, the classic corporate and campus firewall move.

error: RPC failed; curl 92 HTTP/2 stream 0 was not closed cleanly

A proxy or middlebox mishandling HTTP/2 during a large transfer.

fatal: the remote end hung up unexpectedly

The connection died mid-transfer, usually a large push meeting a server or proxy limit.

warning: adding embedded git repository: <folder>

You added a folder that is itself a Git repository; it would be recorded as an empty pointer, confusing every clone.

fatal: in unpopulated submodule '<path>'

You are standing inside a submodule folder that was never initialized, an empty shell.

fatal: no submodule mapping found in .gitmodules for path '<path>'

A submodule folder is recorded in the index but missing from .gitmodules, a half-removed submodule.

error: could not apply <hash>... <message>

A rebase or cherry-pick hit a conflict on this commit and paused for you.

CONFLICT (modify/delete): <file> deleted in <ref> and modified in <ref>

One side deleted the file, the other edited it; Git cannot choose.

remote: Support for password authentication was removed on August 13, 2021.

GitHub no longer accepts account passwords over HTTPS; a token or SSH key is required.

remote: Write access to repository not granted.

Your account can read this repository but not push to it.

fatal: Not a valid object name: 'main'.

Branching from main before any commit exists; a branch cannot point at nothing.



The errors you will meet

The message, then the cause. Every fix is one search away **on the site**.

remote: Permission to <repo> denied to github-actions[bot].

A workflow tried to push, but Actions runs with read-only permissions unless the workflow asks for more.

fatal: your current branch 'main' does not have any commits yet
Log, branch, and friends need at least one commit to talk about.

fatal: shallow update not allowed

You are pushing from a shallow clone, common in CI checkouts.

fatal: Refusing to fetch into current branch
refs/heads/<branch> of non-bare repository

Fetching directly into the branch you are standing on would pull the floor from under you.

error: Server does not allow request for unadvertised object
<hash>

A submodule is pinned to a commit the server will not hand out directly, or a shallow clone wants what it cannot see.

fatal: 'origin/<branch>' is not a commit and a branch '<name>' cannot be created from it

Your clone has never heard of that remote branch; the start point does not exist locally.

error: branch '<name>' not found.

No local branch by that exact name; a typo, or it lives only on the remote.

fatal: There is no merge to abort (MERGE_HEAD missing).
Nothing is mid-merge; whatever state you are in, it is not that.

fatal: No rebase in progress?

The rebase you are trying to continue or abort already ended, or never started.

fatal: could not read Username for 'https://github.com':
terminal prompts disabled

GIT_TERMINAL_PROMPT=0 is set, as in most CI, and Git wanted to ask for credentials.

remote: HTTP Basic: Access denied

GitLab's phrasing of bad HTTPS credentials: an expired or wrong token, or a password where a token belongs.

gpg: signing failed: No secret key

Signing is on, but the configured key is not on this machine, typical after a laptop change.



The errors you will meet

The message, then the cause. Every fix is one search away **on the site**.

gpg: signing failed: Inappropriate ioctl for device

GPG wants to ask for your passphrase but has no terminal to ask in.

fatal: Could not read from remote repository.

The generic tail of every SSH failure; the true cause sits in the lines above it.

fatal: unable to access '<url>': OpenSSL SSL_read: Connection was reset, errno 10054

Something on the network path, a proxy, VPN, or security software, cut the TLS connection, most often on Windows.

fatal: unable to access '<url>': error setting certificate verify locations

Git's CA bundle path is broken, usually after a partial install or an overzealous cleanup.

error: bad signature 0x00000000 (fatal: index file corrupt)

The index file was damaged, a crash, a full disk, or a sync service writing into .git.

fatal: LF would be replaced by CRLF in <file>

The strict form of the line-ending check: safecrlf is set to refuse converting changes instead of warning.

fatal: It seems that there is already a rebase-merge directory

A previous rebase never finished; its state directory still sits in .git.

fatal: unsafe repository ('<path>' is owned by someone else)

The older phrasing of the ownership guard, same cause as dubious ownership.

! [remote rejected] <branch> -> <branch> (pre-receive hook declined)

The server ran a policy check and said no: a protected branch, an oversized file, or a failed lint, and it printed its reason just above this line.

error: GH013: Repository rule violations found for refs/heads/<branch>.

GitHub rulesets blocked the push; the lines above name the exact rule, commonly a required pull request or signed commits.

error: GH006: Protected branch update failed for refs/heads/<branch>.

That branch only accepts changes through reviewed pull requests.

remote: error: File <name> is <size> MB; this exceeds GitHub's file size limit of 100.00 MB

One specific file crossed the hard limit; the push cannot land while any commit carries it.



| The errors you will meet

The message, then the cause. Every fix is one search away **on the site**.

`husky - pre-commit hook exited with code 1 (error)`

The project's own pre-commit checks failed: lint, tests, or formatting, and the real output sits just above.

`Everything up-to-date`

Not an error: push found nothing to send. The surprise usually means the work is uncommitted or on another branch.

`nothing to commit, working tree clean`

Commit found no staged changes. Either everything is already committed, or the edits live elsewhere.

`The requested URL returned error: 403`

The server recognized you and still said no: wrong account's cached credentials, or no write permission here.



The living version

Eight doors, one address: amey-thakur.github.io/GIT-GUIDE

Finder

ask anything, or paste
the error

Start

zero to first push

Learn

the model, step by step

Fix

guided rescue

Errors

Git's messages,
decoded

GitHub

first repo to branch
protection

Workflows

how teams run Git

Cheatsheet

everything, one line
each

SAFE**REWRITES HISTORY****DESTRUCTIVE**

every answer carries its danger level and its undo

Every answer also lives in the repository's Discussions; a question the guide cannot answer becomes its next addition. MIT licensed, no trackers, works offline.

**Amey Thakur**amey-thakur.github.io/GIT-GUIDE

Every Git and GitHub answer in one place.